

SERVERLESS STUDENT FEEDBACK SYSTEM

(A Cloud-Native AWS Serverless Application for Digital Student Feedback Collection)

Prepared in partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr. Juttuka Lovababu, APSSDC

Submitted by

Yenni Ravi Chandrika, A23126552129, (ANITS)

Yalamati Hansika, A23126552125, (ANITS)

Nandisa Naimisha, A23126552106, (ANITS)

Moyyi Hasini, A23126552102, (ANITS)

Mohammed Muskan, A23126552101, (ANITS)

Bandaru Srivalli, A23126552070, (ANITS)

ACKNOWLEDGEMENT

We express our sincere gratitude to the **Andhra Pradesh Skill Development Corporation (APSSDC)** for providing us with the opportunity to participate in the Summer Internship Program on Amazon Web Services (AWS). This internship enabled us to gain practical knowledge of cloud computing, serverless technologies, and modern web application development.

We sincerely thank our mentors, Mrs. Sumana Bethala and Mr. Juttuka Lovababu, for their continuous guidance, valuable suggestions, and encouragement throughout the project. Their expertise and support helped us understand cloud-native application development and successfully complete our project.

We also express our gratitude to the faculty members of the Department of Computer Science and Engineering (Artificial Intelligence & Machine Learning), Anil Neerukonda Institute of Technology and Sciences, for their constant motivation and support during the internship.

We extend our heartfelt thanks to our teammates for their excellent cooperation, dedication, and teamwork, and to everyone who directly or indirectly helped us complete the “Serverless Student Feedback System” successfully.

Sincerely,

Yenni Ravi Chandrika - A23126552129 - ravichandrikay4@gmail.com

Yalamati Hansika - A23126552125 - hansika.yalamati06@gmail.com

Nandisa Naimisha - A23126552106 - nandisanaimisha@gmail.com

Moyyi Hasini – A23126552102 - hasinijaanu145@gmail.com

Mohammed Muskan -A23126552101- muskansalman82@gmail.com

Bandaru Srivalli – A23126552070 - bandarusrivalli4@gmail.com

ABSTRACT

This project presents the design and implementation of a **Serverless Student Feedback System** — a cloud-native web application built entirely on **Amazon Web Services (AWS)** that automates the collection, storage, and analysis of student feedback on faculty and courses. The primary goal is to eliminate the manual, time-consuming, and error-prone tasks associated with paper-based feedback by leveraging serverless compute, managed NoSQL storage, and role-based dashboards.

The system workflow begins when a student logs in, selects a subject, and submits a feedback questionnaire through the web interface. The frontend communicates with the backend through **Amazon API Gateway**, which securely routes each request to **AWS Lambda**. Lambda functions, written in Python, validate the input and store the structured feedback record in Amazon **DynamoDB**. Administrators access a centralized dashboard to view real-time analytics — average ratings, subject-wise distributions, and participation statistics — without performing any manual calculation.

The platform is built on a fully serverless AWS architecture using **API Gateway, Lambda, DynamoDB, S3, IAM, and CloudWatch**, ensuring automatic scaling, high availability, and cost-efficient, pay-per-use operation. **AWS IAM** enforces least-privilege access control between components, while **CloudWatch** provides centralized logging and performance monitoring.

By combining a serverless backend, a responsive HTML5/CSS3/JavaScript frontend, and cloud-native AWS services, the platform demonstrates how modern cloud computing can simplify institutional administrative processes — reducing manual workload, improving data accuracy, and providing real-time, actionable insight into teaching quality.

TABLE OF CONTENTS

S.No	Content	Pg.No
1.	INTRODUCTION.....	5
2.	METHODOLOGY	6
3.	TECHNOLOGY STACK	8
4.	SYSTEM DESIGN/ARCHITECHTURE.....	10
5.	IMPLEMENTATION	13
6.	RESULTS.....	15
7.	CONCLUSION	18

1. INTRODUCTION

In today's educational environment, continuous improvement of teaching quality is essential for providing better learning experiences to students. Educational institutions regularly collect feedback from students to understand faculty performance, identify strengths and weaknesses, and improve academic standards. Traditionally this feedback has been collected using paper-based forms — an approach that suffers from excessive paperwork, manual data entry, delayed report generation, human error during analysis, and high operational cost.

The Serverless Student Feedback System was built to address exactly this problem. It functions as a cloud-native web application that completely digitizes the feedback collection process — from student login and subject selection to submission, storage, and administrator analytics — using AWS serverless technologies including API Gateway, AWS Lambda, Amazon DynamoDB, Amazon S3, IAM, and CloudWatch.

Unlike traditional applications that require dedicated servers, the proposed system follows a fully serverless architecture. AWS Lambda automatically executes backend functions only when requests are received, making the system highly scalable while minimizing operational costs. Amazon DynamoDB stores structured feedback data with low latency, enabling quick retrieval and analysis, while API Gateway securely exposes backend APIs and CloudWatch continuously monitors application performance.

This project was undertaken to:

- Replace the traditional paper-based feedback process with a secure, convenient online web application.
- Enable students to evaluate faculty using predefined questionnaires covering communication, subject knowledge, punctuality, and overall performance.
- Automate backend processing and cloud storage using serverless AWS services, eliminating server management overhead.

- Provide administrators with real-time analytical dashboards, faculty ratings, and statistical reports for academic decision-making.

The remainder of this report describes the methodology followed, the technology stack used, the system design and architecture, the implementation of the feedback pipeline, the results achieved, and the conclusions drawn from the project.

2. METHODOLOGY

The development of the Serverless Student Feedback System followed a structured, iterative methodology comprising requirement analysis, system design, cloud architecture planning, frontend and backend implementation, cloud deployment, and testing. Each phase was aligned with best practices in cloud-native application development.

2.1. Requirement Analysis

The project began with an analysis of the feedback lifecycle to identify recurring bottlenecks in traditional paper-based systems: manual data entry, delayed report generation, lack of transparency, and difficulty scaling to larger student populations. Functional requirements identified include student authentication, subject and faculty selection, feedback submission, cloud data storage, an administrator dashboard, and report generation. Non-functional requirements include scalability, reliability, security, high availability, performance, and cost efficiency.

2.2. System Design

Based on the identified requirements, the overall architecture was designed using AWS serverless services. The application consists of two major components — a frontend where students log in, select subjects, and submit feedback, and a serverless backend consisting of AWS Lambda functions connected through API Gateway that validate inputs, communicate with DynamoDB, and return responses

2.3. Workflow Design

An automated, event-driven feedback workflow was designed as follows: Student Login → Select Subject → Choose Faculty → Answer Feedback Questions → Submit Feedback → Amazon API Gateway → AWS Lambda → Amazon DynamoDB → Administrator Dashboard → Reports & Analytics. Each request from the frontend is securely transmitted to the backend using HTTPS APIs exposed through Amazon API Gateway.

2.4. Security Design

Security was a central design consideration. IAM roles and policies were scoped to least privilege for each component (Lambda execution, API Gateway access, DynamoDB access, and CloudWatch logging), all communication uses HTTPS, and only authenticated users are permitted to submit feedback or access analytical reports.

2.5. Testing & Deployment

Unit, integration, functional, performance, and security testing were performed before deployment. Deployment involved creating Lambda functions, configuring API Gateway, provisioning DynamoDB tables, assigning IAM roles, uploading frontend files to S3, and validating endpoints, with CloudWatch used throughout for log-based verification.

3. TECHNOLOGY STACK

The Serverless Student Feedback System utilizes modern web technologies and AWS managed cloud services to build a scalable and highly reliable application.

3.1. Frontend Technologies

HTML5 provides the structural foundation of the Login Page, Student Dashboard, Feedback Form, and Reports. CSS3 provides responsive styling, ensuring compatibility across desktops, tablets, and mobile devices. JavaScript handles form validation, dynamic content, and asynchronous communication with backend REST APIs.

3.2. Backend Technologies

AWS Lambda serves as the backend execution environment — requiring no server management, scaling automatically, and charging only for actual execution time. Each Lambda function is implemented in Python, chosen for its easy syntax, rich libraries, and strong AWS SDK support.

3.3. API, Database & Storage

Amazon API Gateway exposes secure REST endpoints between the frontend and backend, handling request routing, HTTPS communication, and rate limiting. Amazon DynamoDB, a fully managed NoSQL database, stores all feedback records with low latency and automatic scaling. Amazon S3 hosts the static frontend — HTML, CSS, JavaScript, and image assets — as a durable, low-cost, scalable website.

3.4. Security & Monitoring

AWS IAM manages user authentication, permissions, roles, and policies, ensuring only authorized users and services can access AWS resources. Amazon CloudWatch continuously monitors API requests, Lambda execution, errors, and response times, helping developers identify and resolve issues quickly.

3.5. Technology Stack Summary

Component	Technology Used
Frontend	HTML5, CSS3, JavaScript
Backend	AWS Lambda (Python)
API	Amazon API Gateway
Database	Amazon DynamoDB
Storage	Amazon S3
Authentication	AWS IAM
Monitoring	Amazon CloudWatch
Version Control / IDE	GitHub, Visual Studio Code
Testing	Postman, Web Browser

4. SYSTEM DESIGN / ARCHITECTURE

4.1. Existing System

Most educational institutions still rely on printed feedback forms collected manually at the end of each semester, or on web-based systems that depend on traditional client-server architectures requiring dedicated servers, database administrators, and continuous maintenance. Key limitations of these existing approaches include:

- Paper-based, manual data collection with high operational cost.
- Manual, time-consuming report generation prone to human error.
- Limited scalability and high maintenance requirements.
- Poor security, no automatic backup, and lack of real-time analytics.

4.2. Proposed System

The proposed system follows a fully serverless architecture where backend functions execute only when required, eliminating server management while ensuring automatic scalability, high availability, and reduced operational cost. Students securely log in, select subjects, evaluate faculty, and submit feedback; every request is routed through API Gateway to Lambda, which validates and stores the data in DynamoDB. Administrators access dashboards showing faculty performance, average ratings, and subject-wise feedback distribution.

4.3. AWS Services Used

Amazon API Gateway is the entry point for all client requests, forwarding them to the appropriate Lambda function and handling authentication, CORS, and throttling. AWS Lambda executes backend business logic — login validation, subject and faculty retrieval, feedback submission, and analytics — without any dedicated server. Amazon DynamoDB stores all feedback records with low latency and flexible schema. Amazon S3 hosts the static frontend. AWS IAM enforces fine-grained access control through dedicated execution and access roles. Amazon CloudWatch provides

centralized logging, error tracking, and performance monitoring across every component.

Aws S3:

The screenshot displays the AWS S3 console interface. The top navigation bar shows the AWS logo, a search bar, and the 'Ask Amazon Q' button. The main header indicates the user is in the 'us-east-1' region.

Buckets Page:

- General purpose buckets (1)** Info
- Buttons: Copy ARN, Empty, Delete, Create bucket
- Search: Find buckets by name
- Table:

Name	AWS Region	Creation date
serverless-student-feedback-2026	US East (N. Virginia) us-east-1	July 4, 2026, 17:09:13 (UTC+05:30)

serverless-student-feedback-2026 Details:

- Info
- Objects (9)
- Buttons: Copy S3 URI, Copy URL, Download, Open, Delete, Actions, Create folder, Upload
- Search: Find objects by prefix
- Table:

Name	Type	Last modified	Size	Storage class
admin.css	css	July 13, 2026, 14:52:18 (UTC+05:30)	6.7 KB	Standard
admin.html	html	July 13, 2026, 14:52:18 (UTC+05:30)	6.3 KB	Standard
admin.js	js	July 13, 2026, 14:52:16 (UTC+05:30)	14.5 KB	Standard
index.html	html	July 13, 2026, 14:52:21 (UTC+05:30)	3.4 KB	Standard
login.css	css	July 13, 2026, 14:52:15 (UTC+05:30)	1.3 KB	Standard
login.html	html	July 13, 2026, 14:52:15 (UTC+05:30)	1.1 KB	Standard

CloudShell Feedback | © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Aws DynamoDB:

The screenshot shows the AWS Management Console for the 'StudentFeedback' table in a DynamoDB instance. The left sidebar contains navigation links for DynamoDB, Dashboard, Tables, Explore items, PartiQL editor, Backups, Exports to S3, Imports from S3, Integrations, Reserved capacity, and Settings. The main content area displays the 'StudentFeedback' table overview, including a 'Protect your DynamoDB table from accidental writes and deletes' warning, a 'General information' section with details like Partition key, Sort key, Capacity mode, and Table status, and a 'Get live item count' button.

StudentFeedback (Last updated: July 14, 2026, 11:27 (UTC+5:30))

Settings | **Indexes** | **Monitor** | **Global tables** | **Backups** | **Exports and streams** | **Permissions**

Protect your DynamoDB table from accidental writes and deletes
When you turn on point-in-time recovery (PITR), DynamoDB backs up your table data automatically so that you can restore to any given second in the preceding 1 to 35 days. Additional charges apply. [Learn more](#)

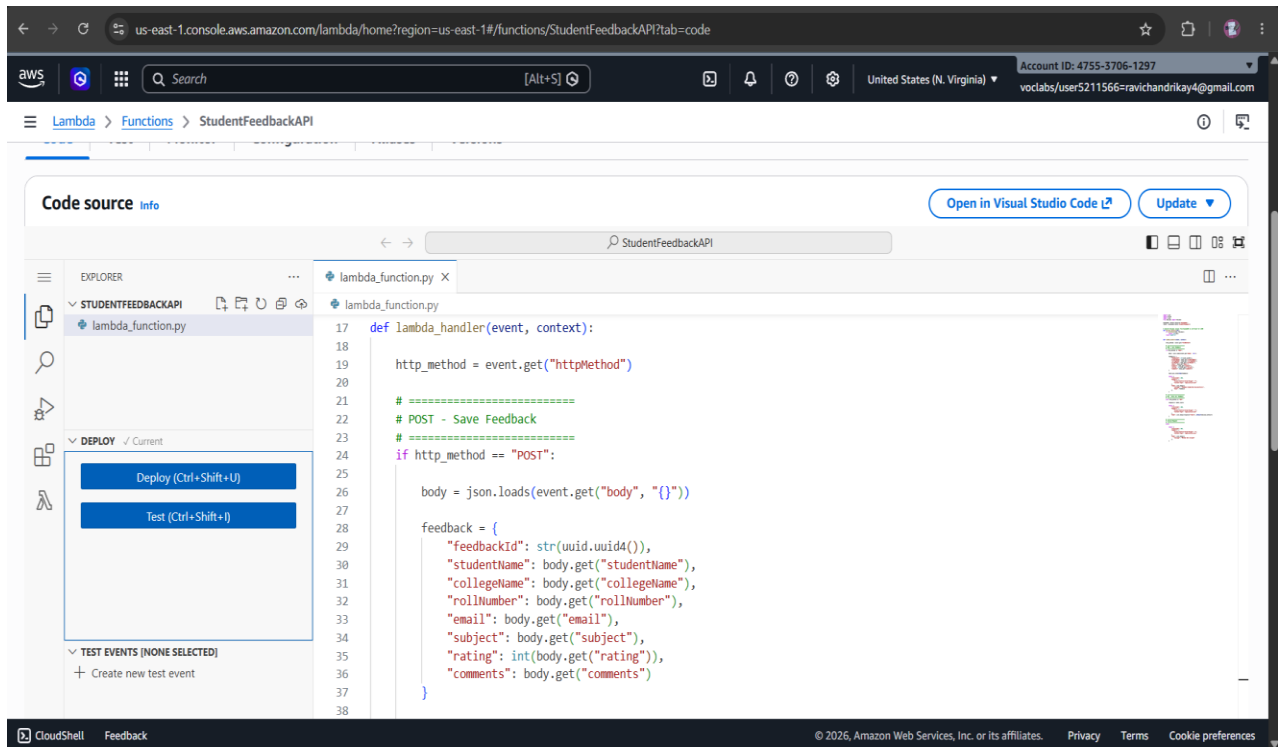
General information

Partition key feedbackId (String)	Sort key -	Capacity mode On-demand	Table status Active
Alarms No active alarms	Point-in-time recovery (PITR) Off	Item count 7	Table size 1.7 kilobytes
Average item size 247.43 bytes	Resource-based policy Not active	Get live item count	
Amazon Resource Name (ARN) arn:aws:dynamodb:us-east-1:475537061297:table/StudentFeedback			

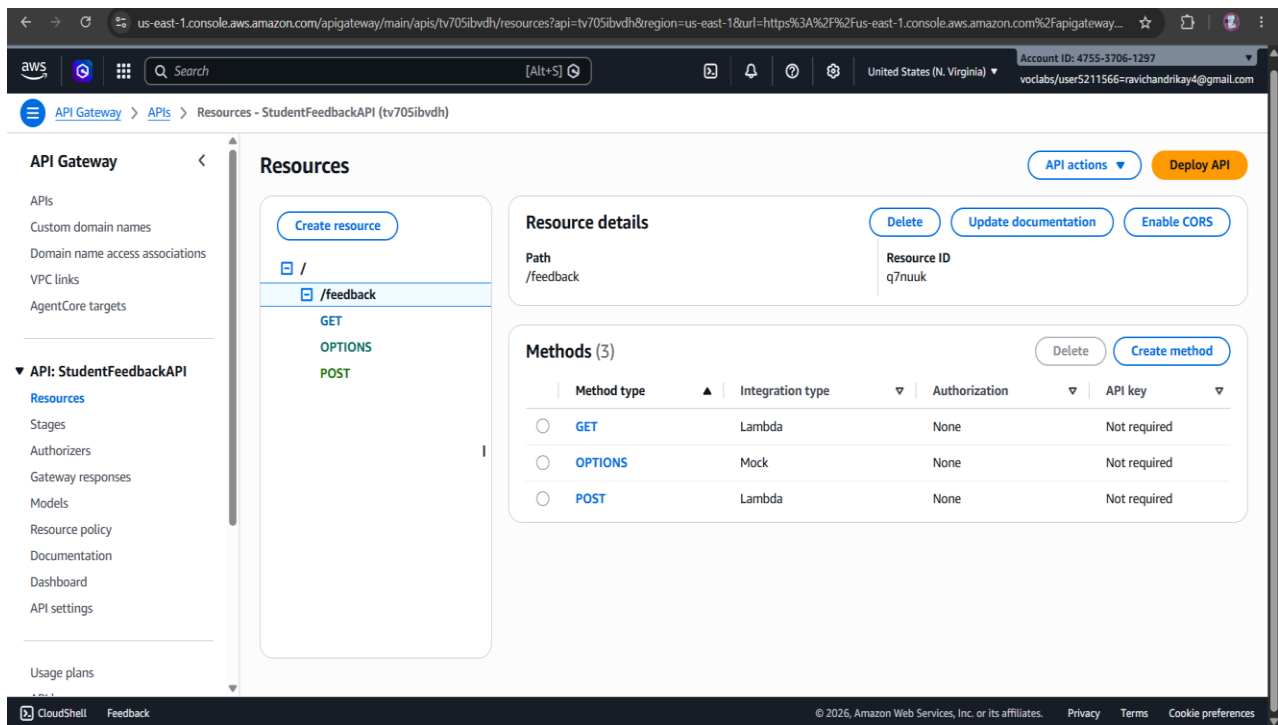
The screenshot shows a web browser displaying a JSON array of feedback data. The URL is 'tv705ibvdh.execute-api.us-east-1.amazonaws.com/prod/feedback'. The JSON array contains five objects, each representing a student's feedback. The objects include fields for subject, collegeName, rollNumber, rating, studentName, comments, feedbackId, and email.

```
[
  {
    "subject": "Mobile Application Development",
    "collegeName": "Anil Neerukonda Institute Of Technology and Sciences",
    "rollNumber": "A23126552125",
    "rating": "4",
    "studentName": "Yalamati Hansika",
    "comments": "It is useful and understanding",
    "feedbackId": "69fbc6e1-67d8-4c19-8ecb-e8ebc7f1d147",
    "email": "honey2006@gmail.com"
  },
  {
    "subject": "Full Stack Development",
    "collegeName": "Anil Neerukonda Institute Of Technology and Sciences",
    "rollNumber": "A23126552106",
    "rating": 4,
    "studentName": "Nandisa Naimisha",
    "comments": "Covered all the Concepts and Gained more Information.",
    "feedbackId": "71ecc0d8-d202-4daa-822d-d3cf4a6bf34d",
    "email": "nandisanaimisha@gmail.com"
  },
  {
    "subject": "Machine Learning",
    "collegeName": "Anil Neerukonda Institute Of Technology and Sciences",
    "rollNumber": "A23126552242",
    "rating": 4,
    "studentName": "Tanigadapa Kavya",
    "comments": "Nice Explaining.",
    "feedbackId": "bbb896ee-c45d-4b53-9b7c-0d820011d2ed",
    "email": "kavya@gmail.com"
  },
  {
    "subject": "Agentic AI",
    "collegeName": "Anil Neerukonda Institute Of Technology and Sciences",
    "rollNumber": "A23126552101",
    "rating": 2,
    "studentName": "Mohammed Muskan",
    "comments": "I didn't get any practical Exposure.",
    "feedbackId": "efa1536c-2fac-4079-a8fb-eda2f119e9fe",
    "email": "muskan@gmail.com"
  },
  {
    "subject": "Cloud Computing",
    "collegeName": "Anil Neerukonda Institute Of Technology and Sciences",
    "rollNumber": "A23126552129",
    "rating": "4"
  }
]
```

Aws Lambda:



Aws API Gateway:



4.4. Architecture Layers

The architecture consists of five layers: a Presentation Layer (Login, Dashboard, Feedback Form, Reports), an API Layer (Amazon API Gateway), a Compute Layer (AWS Lambda functions), a Database Layer (Amazon DynamoDB tables for Students, Faculty, Subjects, and Feedback), and a Monitoring Layer (Amazon CloudWatch). Each layer communicates securely with the next through AWS-managed services, and the full request path is: Student Browser → Amazon S3 (static website) → API Gateway → AWS Lambda → DynamoDB → Administrator Dashboard.

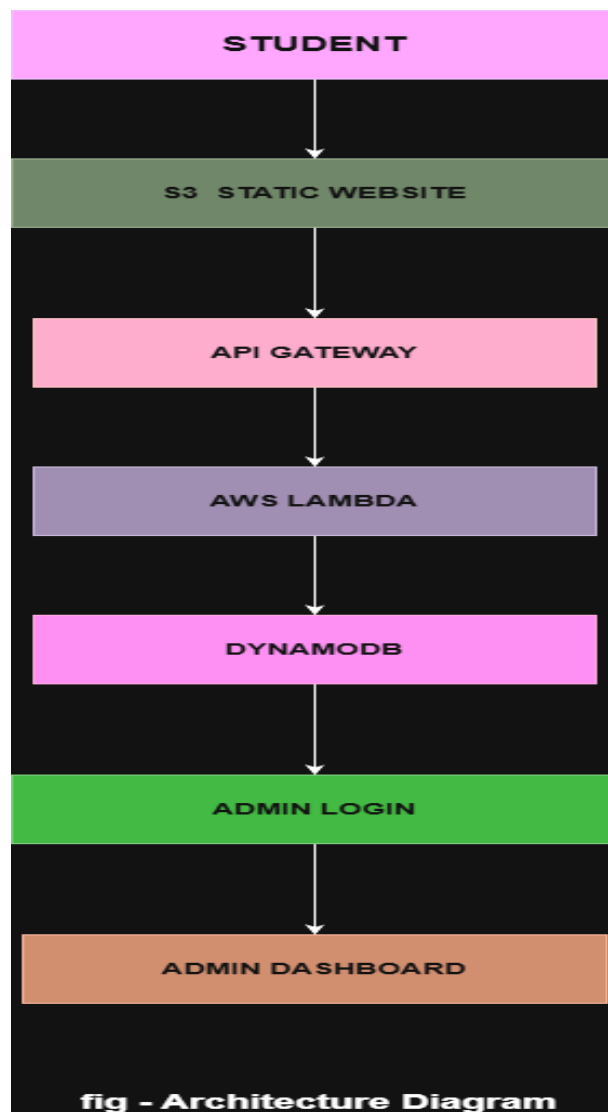


Fig. 4.1 — System Architecture Diagram

4.5. Database Design

Amazon DynamoDB was selected for its fully managed, serverless, automatically-scaling operation with low latency and flexible schema. The database is organized into four entities: a Student table (Student ID, Name, Email, Department, Semester), a Faculty table (Faculty ID, Name, Department, Designation), a Subject table (Subject Code, Subject Name, Semester, Faculty ID), and a Feedback table (Feedback ID, Student ID, Faculty ID, Subject Code, question-wise ratings, Overall Rating, Comments, and Submission Date). One student can submit feedback for multiple subjects, one faculty member can teach multiple subjects, and each feedback record belongs to exactly one student and one faculty member.

5. IMPLEMENTATION

The implementation phase transforms the proposed design into a fully functional web application, integrating the HTML5/CSS3/JavaScript frontend with AWS serverless backend services.

5.1. Frontend Implementation

The frontend consists of a Login Page, Student Dashboard, Subject Selection, Feedback Form, Administrator Dashboard, and Reports Page. CSS provides responsive layouts while JavaScript handles asynchronous communication with backend REST APIs.

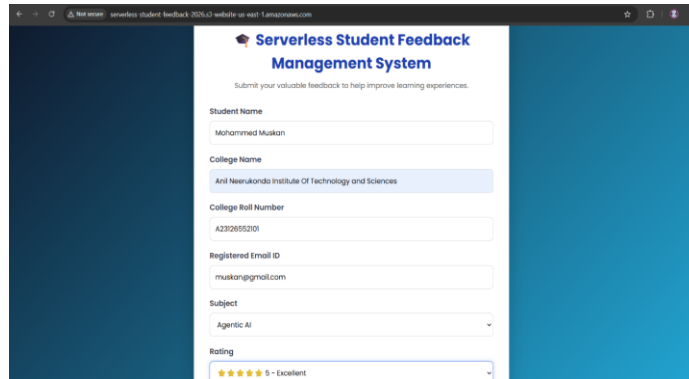
A screenshot of a web browser displaying the 'Serverless Student Feedback Management System' form. The form is titled 'Serverless Student Feedback Management System' with a subtitle 'Submit your valuable feedback to help improve learning experiences.' It contains several input fields: 'Student Name' (filled with 'Mohammed Muskan'), 'College Name' (filled with 'Anil Neerukonda Institute Of Technology and Sciences'), 'College Roll Number' (filled with 'A230265200'), 'Registered Email ID' (filled with 'muskan@gmail.com'), 'Subject' (a dropdown menu showing 'Agentic AI'), and 'Rating' (a star rating system showing 5 stars and the text '5 - Excellent').

Fig. 5.1 — Student Feedback Submission Form

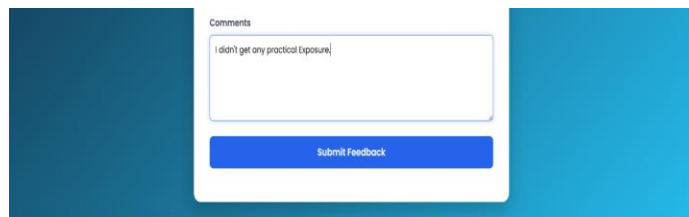
A screenshot of the 'Comments' section of the feedback form. It features a text area with the placeholder text 'i didn't get any practical exposure'. Below the text area is a blue button labeled 'Submit Feedback'.

Fig. 5.2 — Feedback Form: Comments & Submit

5.2. Backend Implementation

The backend consists of AWS Lambda functions written in Python: a Login function that authenticates users and returns session information, a Subject function that retrieves available subjects, a Faculty function that fetches faculty tied to a subject, a Feedback function that validates and stores submitted feedback, and an Analytics function that calculates faculty ratings and generates reports.

5.3. API Integration & Feedback Submission Process

Amazon API Gateway connects the frontend with backend Lambda functions along the path Frontend → API Gateway → Lambda → DynamoDB → Response. When a student logs in, selects a subject, rates their faculty, and clicks Submit, API Gateway receives the request, Lambda validates it, the record is written to DynamoDB, and a success response confirms the submission.

5.4. Admin Login & Dashboard

Administrators authenticate through a dedicated login page before accessing the dashboard, which displays Total Students, Total Faculty, Total Subjects, Total Feedback Submitted, Average Ratings, and Faculty Rankings, all retrieved dynamically from DynamoDB.

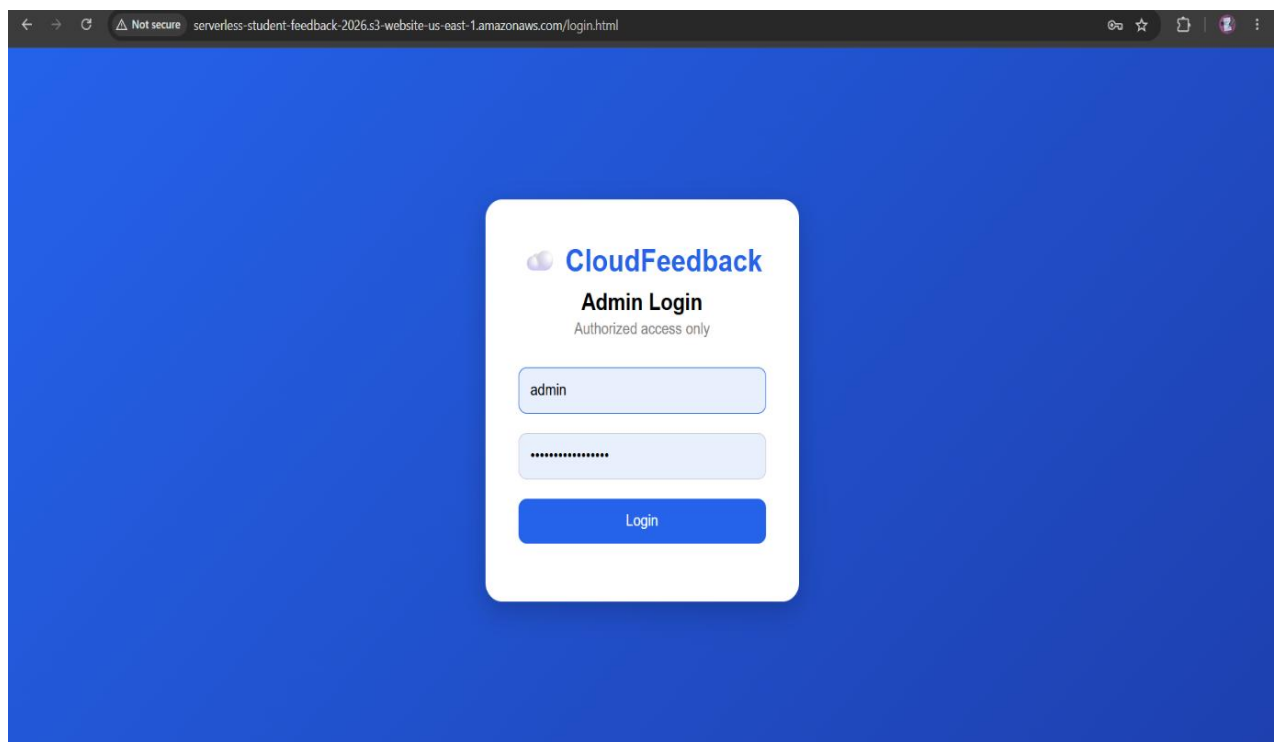


Fig. 5.3 — Admin Login Page

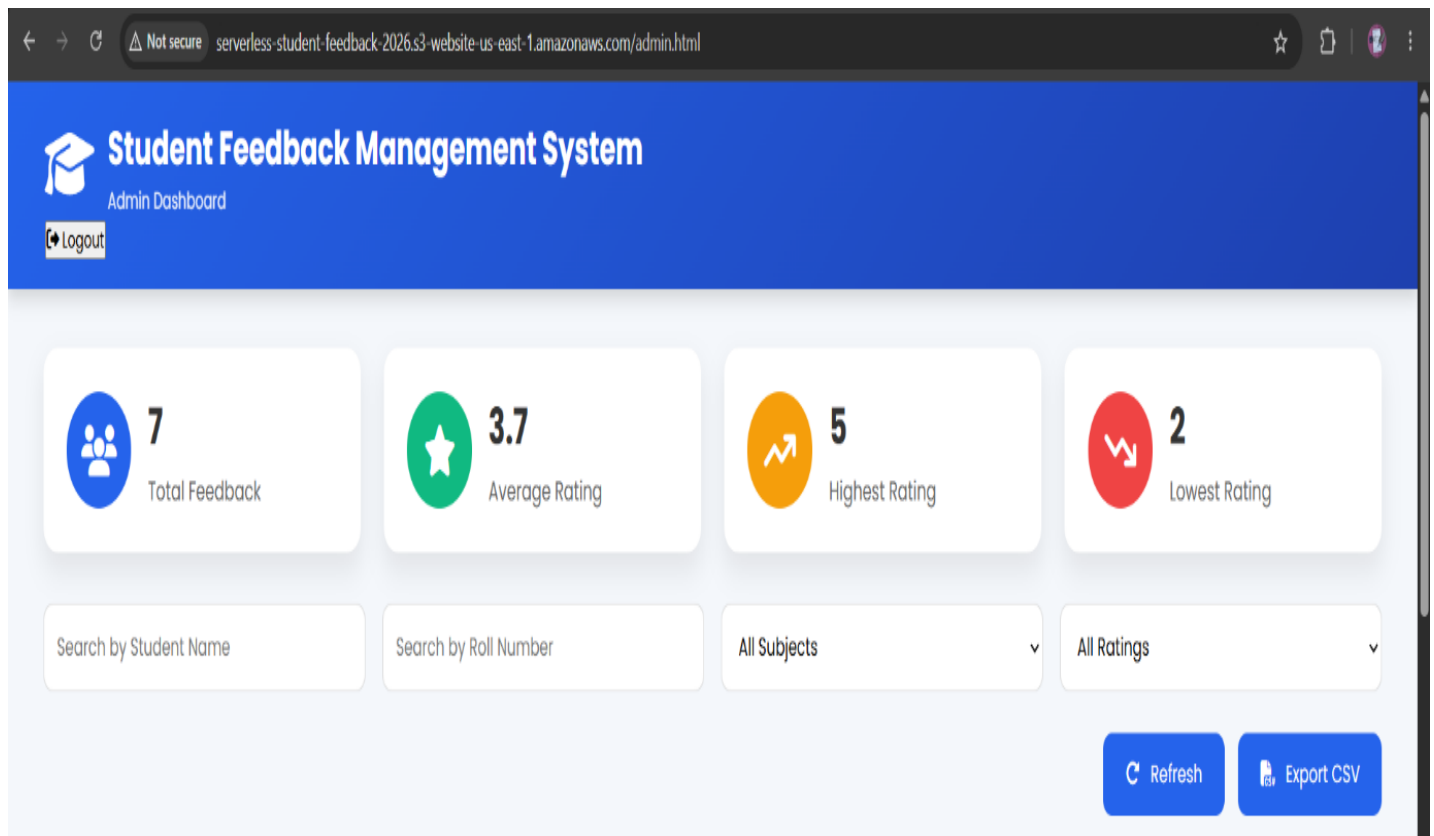


Fig. 5.4 — Administrator Dashboard Overview

5.5. Error Handling & Security

The system handles invalid login attempts, missing mandatory feedback fields, database errors, and API errors gracefully, returning meaningful messages to users while logging details to CloudWatch. Security measures include IAM roles and policies, HTTPS communication, API authorization, and controlled, least-privilege resource access.

5.6. Deployment

Deployment involved uploading frontend files to Amazon S3, creating Lambda functions, configuring API Gateway, provisioning DynamoDB tables, assigning IAM permissions, testing every API endpoint, and monitoring logs through CloudWatch. The deployed application remains accessible through a secure web URL.

6. RESULTS

The Serverless Student Feedback System was successfully developed and deployed using AWS serverless technologies, and tested across multiple scenarios to evaluate functionality, performance, reliability, security, and scalability.

6.1. Successful Authentication & Feedback Submission

The login module was validated using valid and invalid credentials — only authenticated users were granted access, with unauthorized attempts correctly rejected. Students were able to select subjects, evaluate faculty, provide ratings, and submit feedback successfully, with every submitted record stored accurately in DynamoDB and a confirmation message shown on completion.

6.2. Reliable Cloud Infrastructure

Amazon DynamoDB delivered low latency and automatic scalability throughout testing. Amazon API Gateway routed all requests to Lambda with minimal delay and remained responsive under multiple consecutive requests. AWS Lambda functions executed independently and scaled automatically according to workload, with no server management required at any point.

6.3. Administrator Dashboard & Analytics

The administrator dashboard displayed real-time totals for students, subjects, and feedback submitted, along with faculty ratings and subject-wise reports — all generated dynamically from DynamoDB without manual calculation.

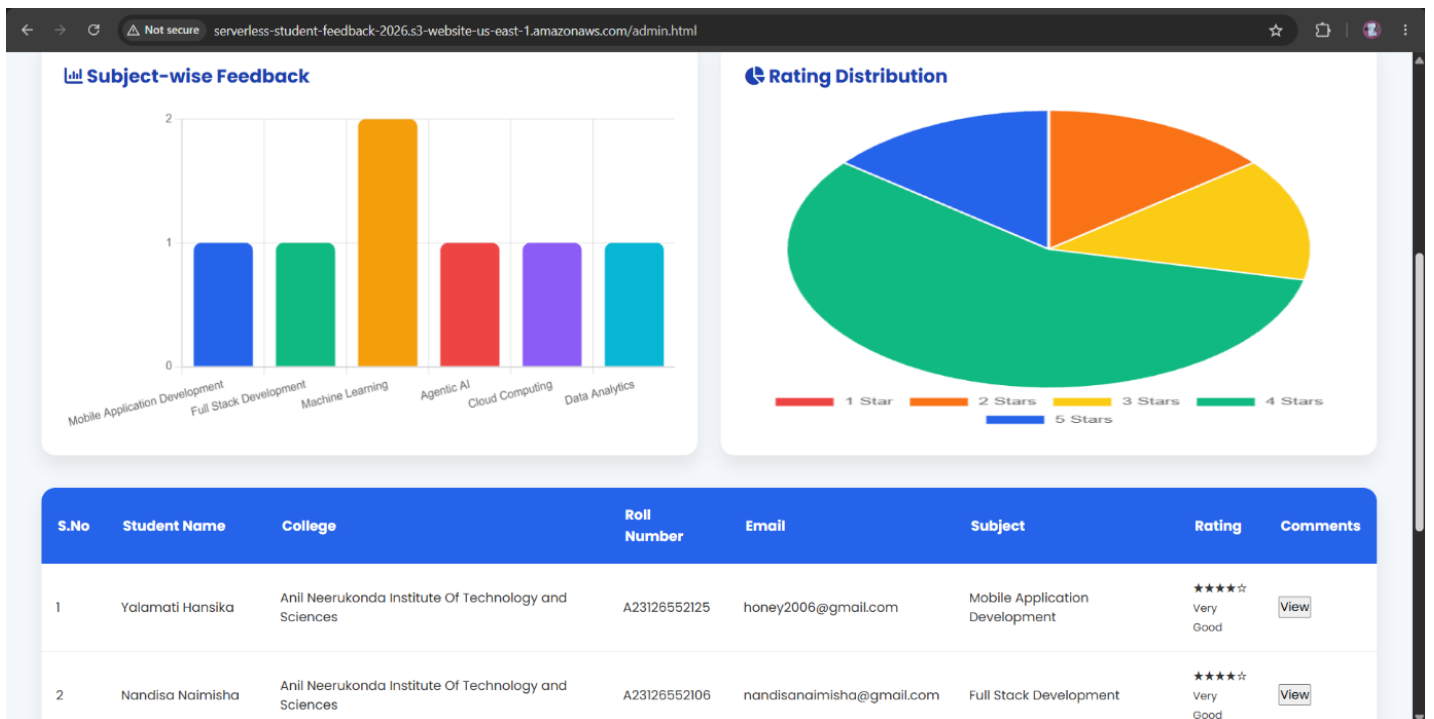


Fig. 6.1 — Subject-wise Feedback & Rating Distribution Charts

5	Yenni Ravi Chandrika	Anil Neerukonda Institute Of Technology and Sciences	A23126552129	ravichandrikay4@gmail.com	Cloud Computing	★★★★☆ Very Good	View
6	Ravi Chandrika	ABC Engineering College	22A91A05A1	ravi@example.com	Machine Learning	★★★★★ Excellent	View
7	Moyya Hasini	Anil Neerukonda Institute Of Technology and Sciences	A23126552102	hasini2006@gmail.com	Data Analytics	★★★★☆ Good	View

Student Feedback Details	
Student Name	Yalamati Hansika
College	Anil Neerukonda Institute Of Technology and Sciences
Roll Number	A23126552125
Email	honey2006@gmail.com
Subject	Mobile Application Development
Rating	4 ★ (Very Good)
Comments	It is useful and understanding

Fig. 6.2 — Student Feedback Records & Details

6.4. Monitoring, Scalability & Security

Amazon CloudWatch provided centralized logging of Lambda execution, API requests, and errors, simplifying debugging. The system scaled automatically under simultaneous requests with no manual intervention, and security testing confirmed that IAM policies and HTTPS communication successfully prevented unauthorized access to feedback data and reports.

6.5. Comparative Analysis

Parameter	Existing System	Proposed System
Feedback Collection	Paper Based	Online
Processing	Manual	Automatic
Report Generation	Manual	Real-Time
Infrastructure	Traditional Servers	AWS Serverless
Database	Local Storage	Amazon DynamoDB
Security	Limited	IAM + HTTPS
Scalability	Limited	Automatic
Maintenance	High	Very Low
Cost	High	Pay-as-you-Go
Monitoring	Manual	Amazon CloudWatch

6.6. Overall Result

The Serverless Student Feedback System successfully achieved all objectives defined during project planning — automating feedback collection, eliminating manual intervention, ensuring secure cloud-based data management, and providing administrators with real-time analytical reports. The integration of AWS serverless technologies resulted in a scalable, reliable, and cost-effective solution suitable for deployment in educational institutions of different sizes.

7. CONCLUSION

The Serverless Student Feedback System was successfully designed, developed, and implemented as a cloud-based application using Amazon Web Services. The project demonstrates how modern serverless computing can simplify and improve the traditional process of collecting, managing, and analyzing student feedback — replacing manual, paper-based methods with an automated digital platform that enhances efficiency, accuracy, transparency, and accessibility.

The application integrates Amazon API Gateway, AWS Lambda, Amazon DynamoDB, Amazon S3, AWS IAM, and Amazon CloudWatch to create a scalable and secure serverless architecture, providing seamless frontend-to-backend communication, secure data storage, automatic request processing, and efficient cloud resource management. One of the project's major strengths is its serverless design: backend operations execute only when requests are received, reducing operational cost, eliminating server administration, and enabling automatic scaling based on demand.

Throughout development, emphasis was placed on security, reliability, and performance — AWS IAM managed permissions and secure access, HTTPS protected all data in transit, and CloudWatch continuously monitored performance and logs. Testing confirmed that every functional module, including login, subject selection, feedback submission, report generation, and dashboard analytics, operated successfully under different scenarios.

Although the current implementation successfully achieves its primary objectives, several opportunities exist for future enhancement, including AI-driven sentiment analysis of comments, machine-learning-based predictive analytics, a dedicated mobile application, multilingual support, and integration with institutional ERP and Learning Management Systems.

In conclusion, the Serverless Student Feedback System effectively demonstrates the practical application of cloud computing and serverless technologies in the education sector, fulfilling the requirements of the AWS Summer Internship Program while serving as a strong foundation for future research and large-scale deployment in educational institutions.